Lab 19

Writing a Custom Handler

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs and paths in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **`<APIM_HOST>`** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **`<GW_HTTPS_PORT>`** – API Gateway HTTPS port: 8243 plus any port offset.
- **`<APIM_HOME>`** – the folder where API Manager is extracted.

The server uses a self-signed certificate by default. The -k option in the curl commands below skips the certificate check for that reason.

Training Objective

Learn how to write a custom API Gateway handler in Java, build it, and engage it for one API by editing the API template.

Business Scenario

PizzaShack wants a simple custom check in front of its PizzaShack API: only internal callers that send a Username header starting with "admin" may call it. This is a teaching example of a custom authentication handler; for this one API it replaces the normal OAuth2 check.

High-Level Steps

- Write and build the custom handler
- Engage the handler for the PizzaShack API
- Test the handler, then clean up

Prerequisites

- The PizzaShack API is deployed and published (context `/pizzashack`, version `1.0.0`).
- JDK 21 and Apache Maven 3.x are installed (`java -version` and `mvn -v` work in a terminal).
- Internet access for the first build: Maven downloads two API Manager libraries from the WSO2 public Maven repository.
- Lab files: the `pizzashack-custom-handler` Maven project (`pom.xml` and `UsernameHeaderHandler.java`) and a ready-built `pizzashack-custom-handler-1.0.jar`.

Part 1 – Write and Build the Handler

1. Understand the handler.

A handler extends `org.apache.synapse.rest.AbstractHandler` and implements `handleRequest` and `handleResponse`. Returning `true` passes the request on; returning `false` stops it. This handler reads the `Username` header and lets the call through only when it starts with `admin`; otherwise it sends HTTP 401 back to the client.

CC BY 4.0

2. Create the Maven project folders. In a working folder, create this structure:

```
pizzashack-custom-handler/
├── pom.xml
└── src/
    └── main/
        └── java/
            └── org/
                └── pizzashack/
                    └── handler/
                        └── UsernameHeaderHandler.java
```

The lab files contain this project ready to use. To build it yourself, create the two files in the next steps.

3. Create `pom.xml` in the `pizzashack-custom-handler` folder:

The two dependencies are the API Manager libraries the handler uses, at the same versions as the jars in `<APIM_HOME>/repository/components/plugins`. They are marked `provided` because API Manager already has them, so they are not packed into the jar.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.pizzashack</groupId>
    <artifactId>pizzashack-custom-handler</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <synapse.version>4.1.0.wso2v40</synapse.version>
        <axis2.version>1.6.1.wso2v125</axis2.version>
    </properties>

    <repositories>
        <repository>
            <id>wso2-nexus</id>
            <name>WSO2 public repository</name>
            <url>https://maven.wso2.org/nexus/content/groups/wso2-public/</url>
        </repository>
    </repositories>

    <dependencies>
        <dependency>
            <groupId>org.apache.synapse</groupId>
            <artifactId>synapse-core</artifactId>
            <version>${synapse.version}</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>org.apache.axis2.wso2</groupId>
            <artifactId>axis2</artifactId>
            <version>${axis2.version}</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <finalName>pizzashack-custom-handler-1.0</finalName>
    </build>
</project>
```

CC BY 4.0

4. Create `UsernameHeaderHandler.java` in `src/main/java/org/pizzashack/handler`:

UsernameHeaderHandler.java

```
package org.pizzashack.handler;

import java.util.Map;

import org.apache.synapse.MessageContext;
import org.apache.synapse.core.axis2.Axis2MessageContext;
import org.apache.synapse.core.axis2.Axis2Sender;
import org.apache.synapse.rest.AbstractHandler;

/**
 * Sample API Gateway handler for the training lab.
 * Lets a request through only when its "Username" header starts with "admin".
 * Any other request is answered with HTTP 401 and is not sent to the back end.
 */
public class UsernameHeaderHandler extends AbstractHandler {

    private static final String HEADER = "Username";

    @Override
    public boolean handleRequest(MessageContext messageContext) {
        org.apache.axis2.context.MessageContext axis2Ctx =
                ((Axis2MessageContext) messageContext).getAxis2MessageContext();
        Map<?, ?> headers = (Map<?, ?>) axis2Ctx.getProperty(
                org.apache.axis2.context.MessageContext.TRANSPORT_HEADERS);
        Object user = headers == null ? null : headers.get(HEADER);
        System.out.println("UsernameHeaderHandler: " + HEADER + " = " + user);
        if (user != null && user.toString().startsWith("admin")) {
            return true;
        }
        // Reject: send a 401 with no body back to the client
        // instead of calling the back end.
        axis2Ctx.setProperty("HTTP_SC", 401);
        axis2Ctx.setProperty("NO_ENTITY_BODY", Boolean.TRUE);
        messageContext.setProperty("RESPONSE", "true");
        messageContext.setTo(null);
        Axis2Sender.sendBack(messageContext);
        return false;
    }

    @Override
    public boolean handleResponse(MessageContext messageContext) {
        return true;
    }
}
```

5. Build the project. Open a terminal in the `pizzashack-custom-handler` folder (the folder that contains `pom.xml`) and run:

All platforms

```
mvn clean install
```

Expected: the build ends with `BUILD SUCCESS` and creates the jar:

```
target/pizzashack-custom-handler-1.0.jar
```

The first build takes longer because Maven downloads the dependencies. If it fails with a download error, check your internet or proxy settings. Make sure `mvn -v` reports Java 21.

Using IntelliJ IDEA: open the `pizzashack-custom-handler` folder as a project (File → Open), set the project SDK to JDK 21, then in the Maven tool window double-click **Lifecycle → clean** and then **install**.

CC BY 4.0

6. Copy the jar into API Manager. A ready-built copy is also in the lab files.

macOS / Linux

```
cp target/pizzashack-custom-handler-1.0.jar <APIM_HOME>/repository/components/lib/
```

Windows

```
copy target\pizzashack-custom-handler-1.0.jar <APIM_HOME>\repository\components\lib\
```

Part 2 – Engage the Handler for the PizzaShack API

7. Back up the API template. Every API's gateway configuration is generated from this file, so edit it with care.

macOS / Linux

```
cp <APIM_HOME>/repository/resources/api_templates/velocity_template.xml \
   <APIM_HOME>/repository/resources/api_templates/velocity_template.xml.bak
```

Windows (Command Prompt)

```
copy ^
  <APIM_HOME>\repository\resources\api_templates\velocity_template.xml ^
  <APIM_HOME>\repository\resources\api_templates\velocity_template.xml.bak
```

8. Open `velocity_template.xml` in a text editor and search for `<handlers xmlns`. The original block looks like this:

Before (original)

```
<handlers xmlns="http://ws.apache.org/ns/synapse">
#foreach( $handler in $handlers )
<handler xmlns="http://ws.apache.org/ns/synapse" class="$handler.className">
                    #if($handler.hasProperties())
                        #set ($map = $handler.getProperties() )
                        #foreach($property in $map.entrySet())
    <property name="$!property.key" value="$!property.value"/>
                        #end
                    #end
</handler>
               #end
```

CC BY 4.0

9. Replace that block with the one below. Seven lines are new: the three #set lines, the #if / handler / #end lines before the loop, and the #if(!(... line with its #end inside the loop. Everything else stays the same.

After (copy this block)

```
<handlers xmlns="http://ws.apache.org/ns/synapse">
#set( $pizzaApi = $apiContext.startsWith("/pizzashack") )
#set( $authHandler = "org.wso2.carbon.apimgt.gateway.handlers.security.APIAuthenticationHandler"
)
#set( $throttleHandler = "org.wso2.carbon.apimgt.gateway.handlers.throttling.ThrottleHandler" )
#if( $pizzaApi )
<handler class="org.pizzashack.handler.UsernameHeaderHandler"/>
#end
#foreach( $handler in $handlers )
#if( !($pizzaApi && ($handler.className == $authHandler || $handler.className ==
$throttleHandler)) )
<handler xmlns="http://ws.apache.org/ns/synapse" class="$handler.className">
                    #if($handler.hasProperties())
                        #set ($map = $handler.getProperties() )
                        #foreach($property in $map.entrySet())
    <property name="$!property.key" value="$!property.value"/>
                        #end
                    #end
</handler>
#end
               #end
```

What the new lines do:

- The three `#set` lines remember whether this API's context starts with `/pizzashack` and name the two standard handlers to leave out.
- The first `#if … #end` adds the custom handler, only for PizzaShack.
- The second `#if … #end` (inside the loop) leaves out the standard authentication and throttling handlers, only for PizzaShack. All other APIs keep their normal handlers.

Save the file. Leave the rest of the template unchanged.

10. Restart API Manager so that it loads the new jar and the edited template.

macOS / Linux

```
<APIM_HOME>/bin/api-manager.sh stop
<APIM_HOME>/bin/api-manager.sh start
```

Windows (stop the running server with Ctrl+C first)

```
<APIM_HOME>\bin\api-manager.bat
```

11. In the Publisher, open the PizzaShack API, go to **Deploy → Deployments** and click **Deploy New Revision**. The gateway configuration of the API is generated again from the edited template.

Part 3 – Test the Handler

12. Call the API with the Username header set to admin. No access token is needed now for this API.

Request 1 – allowed

```
curl -k -i \
  -H "accept: application/json" \
  -H "Username: admin" \
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu
```

CC BY 4.0

Expected: `HTTP/1.1 200 OK` and the pizza menu in JSON.

13. Call the API with a different user name:

Request 2 – rejected

```
curl -k -i \
  -H "accept: application/json" \
  -H "Username: kamal" \
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu
```

Expected: `HTTP/1.1 401 Unauthorized` with no body. A call without the `Username` header also returns 401.

14. Check the API Manager console (the terminal window where API Manager is running). For each call you see a line such as:

```
UsernameHeaderHandler: Username = admin
UsernameHeaderHandler: Username = kamal
```

The handler writes with `System.out`, so these lines appear only in the console, not in `wso2carbon.log`. Other APIs are not affected and still require a normal access token.

On Windows Command Prompt, replace the `\` at the end of each line with `^`, or put the command on one line.

Part 4 – Clean Up

15. After the lab, put API Manager back to normal:

- Restore the original template: copy `velocity_template.xml.bak` back over `velocity_template.xml`.
- Remove `pizzashack-custom-handler-1.0.jar` from `<APIM_HOME>/repository/components/lib`.
- Restart API Manager.
- Deploy a new revision of the PizzaShack API.

Expected Outcome

A custom handler written in Java is engaged for the PizzaShack API through the API template. Calls are accepted (HTTP 200) or rejected (HTTP 401) by the handler instead of the standard OAuth2 check, while all other APIs are unchanged.

CC BY 4.0